Lab 6: Code and Branch Coverage Analysis

Atharv Mittal | PES1UG24CS091

Final results

Stage	Tests passed	Final application coverage	Requirement
Code coverage	12	39 / 39 statements = 100%	Minimum 90% met
Branch coverage	50	26 / 26 branches = 100%	Minimum 90% met

The branch run also achieves 100% line coverage, with no missing statements or partial branches. Only spam_scoring.py is included in the coverage measurement. The supplied application source is unchanged.

Submission verification

- Separate Python files are maintained for code coverage and branch coverage; their full source is included in this report.
- Both final terminal screenshots are embedded in this report and show 100% coverage.
- Both test files and both screenshots are included in the pushed Lab-6 folder.

Repository and commit evidence

GitHub repository: AtharvMitt/digital-gift-card-loyalty-exchange

ec88202 — Code coverage tests and laptop screenshot

b5bf2f3 — Branch coverage tests and laptop screenshot

1. Supplied application

File: spam_scoring.py. The module scores messages, classifies scores, counts inbox labels and determines inbox status.

```
SPAM_WORDS = {"free", "winner", "prize", "urgent", "cash"}


def score_message(text, sender_known, link_count):
    score = 0
    for word in text.split():
        if word.lower().strip("!.,") in SPAM_WORDS:
            score += 2
        elif word.isupper() and len(word) > 3:
            score += 1
    if link_count > 3:
        score += 4
    elif link_count > 0:
        score += 1
    if sender_known:
        score -= 3
        if score < 0:
            score = 0
    return score


def classify(score):
    if score >= 8:
        return "spam"
    if score >= 4:
        return "suspicious"
    return "clean"


def filter_inbox(messages):
    counts = {"spam": 0, "suspicious": 0, "clean": 0}
    for text, known, links in messages:
        label = classify(score_message(text, known, links))
        counts[label] += 1
    return counts


def inbox_status(counts):
    total = sum(counts.values())
    if total == 0:
        return "empty"
    ratio = counts["spam"] / total
    if ratio > 0.5:
        return "flooded"
    if ratio > 0.2:
        return "noisy"
    return "healthy"
```

2. Code coverage test cases

File: CodeCoverageTest_PES1UG24CS091_spam_scoring.py

Result: 12 passed; 39 statements executed out of 39; 100% code coverage.

```
"""Stage 1: exercise every executable line of the supplied application."""

import pytest

from spam_scoring import classify, filter_inbox, inbox_status, score_message


def test_plain_message_scores_zero():
    assert score_message("hello there", False, 0) == 0


def test_spam_words_and_uppercase_with_many_links():
    assert score_message("FREE! winner, HELLO", False, 4) == 9


def test_some_links_add_one():
    assert score_message("hello", False, 1) == 1


def test_known_sender_score_is_clamped_at_zero():
    assert score_message("hello", True, 0) == 0


@pytest.mark.parametrize("score, expected", [(1, "clean"), (4, "suspicious"), (8, "spam")])
def test_classification(score, expected):
    assert classify(score) == expected


def test_inbox_counts_all_three_labels():
    messages = [("hello", False, 0), ("free cash", False, 0), ("free cash", False, 4)]
    assert filter_inbox(messages) == {"spam": 1, "suspicious": 1, "clean": 1}


@pytest.mark.parametrize(
    "counts, expected",
    [
        ({"spam": 0, "suspicious": 0, "clean": 0}, "empty"),
        ({"spam": 3, "suspicious": 0, "clean": 1}, "flooded"),
        ({"spam": 1, "suspicious": 1, "clean": 1}, "noisy"),
        ({"spam": 0, "suspicious": 1, "clean": 3}, "healthy"),
    ],
)
def test_inbox_status(counts, expected):
    assert inbox_status(counts) == expected
```

3. Branch coverage test cases — part 1

File: BranchCoverageTest_PES1UG24CS091_spam_scoring.py

Result: 50 passed; 26 branch destinations executed out of 26; 100% branch coverage. This independent suite includes scoring and classification thresholds.

```
"""Stage 2: decision outcomes, thresholds, empty inputs and integration."""

import pytest

from spam_scoring import classify, filter_inbox, inbox_status, score_message


@pytest.mark.parametrize(
    "text, known, links, expected",
    [
        ("", False, 0, 0),                      # zero loop iterations
        ("   \t\n", False, 0, 0),
        ("hello there", False, 0, 0),             # ordinary words
        ("free", False, 0, 2),
        ("winner", False, 0, 2),
        ("prize", False, 0, 2),
        ("urgent", False, 0, 2),
        ("cash", False, 0, 2),
        ("FrEe! WINNER, prize. urgent cash", False, 0, 10),
        ("free free", False, 0, 4),               # count each occurrence
        ("FREE", False, 0, 2),                    # keyword precedes uppercase rule
        ("HELLO", False, 0, 1),
        ("HEY", False, 0, 0),                     # uppercase length == 3
        ("HEYA", False, 0, 1),                    # uppercase length == 4
        ("Hello", False, 0, 0),                   # not all uppercase
        ("1234", False, 0, 0),
        ("hello", False, 1, 1),                   # first positive link count
        ("hello", False, 3, 1),                   # upper end of small-link range
        ("hello", False, 4, 4),                   # first large-link count
        ("hello", True, 0, 0),                    # negative adjusted score clamps
        ("free", True, 0, 0),                     # 2 - 3 clamps
        ("FREE HELLO", True, 0, 0),               # 3 - 3: clamp is skipped
        ("free cash", True, 0, 1),                # positive score skips clamp
        ("FREE! winner, HELLO", False, 4, 9),
        ("FREE! winner, HELLO", True, 4, 6),      # sender deduction after links
    ],
)
def test_score_decisions_and_boundaries(text, known, links, expected):
    assert score_message(text, known, links) == expected


@pytest.mark.parametrize(
    "score, expected",
    [(0, "clean"), (1, "clean"), (3, "clean"), (4, "suspicious"),
     (5, "suspicious"), (7, "suspicious"), (8, "spam"), (9, "spam")],
)
def test_classification_thresholds(score, expected):
    assert classify(score) == expected
```

3. Branch coverage test cases — part 2

Continuation of the same branch test file: inbox counts, status boundaries and integration.

```
@pytest.mark.parametrize(
    "messages, expected",
    [
        ([], {"spam": 0, "suspicious": 0, "clean": 0}),
        ([("hello", False, 0)], {"spam": 0, "suspicious": 0, "clean": 1}),
        ([("free cash", False, 0)], {"spam": 0, "suspicious": 1, "clean": 0}),
        ([("free cash", False, 4)], {"spam": 1, "suspicious": 0, "clean": 0}),
        ([("hello", False, 0), ("free cash", False, 0), ("free cash", False, 4)],
         {"spam": 1, "suspicious": 1, "clean": 1}),
        ([("free cash", False, 4), ("free cash", True, 4)],
         {"spam": 1, "suspicious": 1, "clean": 0}),
    ],
)
def test_inbox_loop_and_counts(messages, expected):
    original = list(messages)
    counts = filter_inbox(messages)
    assert counts == expected
    assert sum(counts.values()) == len(messages)
    assert messages == original


@pytest.mark.parametrize(
    "counts, expected",
    [
        ({"spam": 0, "suspicious": 0, "clean": 0}, "empty"),
        ({"spam": 0, "suspicious": 0, "clean": 1}, "healthy"),
        ({"spam": 1, "suspicious": 2, "clean": 3}, "healthy"),  # below .2
        ({"spam": 1, "suspicious": 2, "clean": 2}, "healthy"),  # exactly .2
        ({"spam": 1, "suspicious": 1, "clean": 2}, "noisy"),    # above .2
        ({"spam": 2, "suspicious": 0, "clean": 3}, "noisy"),    # below .5
        ({"spam": 1, "suspicious": 0, "clean": 1}, "noisy"),    # exactly .5
        ({"spam": 3, "suspicious": 1, "clean": 1}, "flooded"),  # above .5
        ({"spam": 1, "suspicious": 0, "clean": 0}, "flooded"),
        ({"spam": 0, "suspicious": 5, "clean": 0}, "healthy"),
    ],
)
def test_status_thresholds(counts, expected):
    original = dict(counts)
    assert inbox_status(counts) == expected
    assert counts == original


def test_filter_and_status_integration():
    messages = [("free cash", False, 4)] * 3 + [("hello", True, 0)]
    assert inbox_status(filter_inbox(messages)) == "flooded"
    assert inbox_status(filter_inbox([])) == "empty"
```

4. Coverage analysis and reproduction

Initial results

The two original starter tests pass. Initial line coverage is 16/39 statements (41.03%). Initial branch coverage is 9/26 branch destinations (34.62%). The combined coverage.py branch-run percentage is 38.46% (displayed as 38%). Initial results are preserved as text; only final results above 90% are included as screenshots.

How the coverage improved

The code coverage suite exercises spam keywords, uppercase words, each link scoring block, the known sender clamp, all three classifications, inbox counting and all four inbox statuses. It achieves 100% line coverage.

A branch run of the line suite still misses the transition from line 17 to line 19: a known sender whose adjusted score is nonnegative skips the clamp. It covers 25/26 branches (96.15%), while its combined line-and-branch coverage is displayed as 98%. This illustrates why line coverage alone does not establish complete branch coverage.

The branch suite explicitly tests negative, zero and positive adjusted sender scores. It also checks empty messages and inboxes, all keywords, uppercase word lengths 3 and 4, link counts 0/1/3/4, scores around 4 and 8, and spam ratios below, at and above 0.2 and 0.5. At exactly 20% spam the status is healthy; at exactly 50% it is noisy, matching the strict > comparisons. All 26 branch destinations are covered.

Coverage concepts

Line coverage measures executable statements reached. Branch coverage measures alternate control-flow destinations taken. Function coverage asks which functions were called; all four application functions are exercised here. Path coverage considers complete execution sequences. Full branch coverage does not prove that every possible loop path or input combination has been tested.

Commands to reproduce

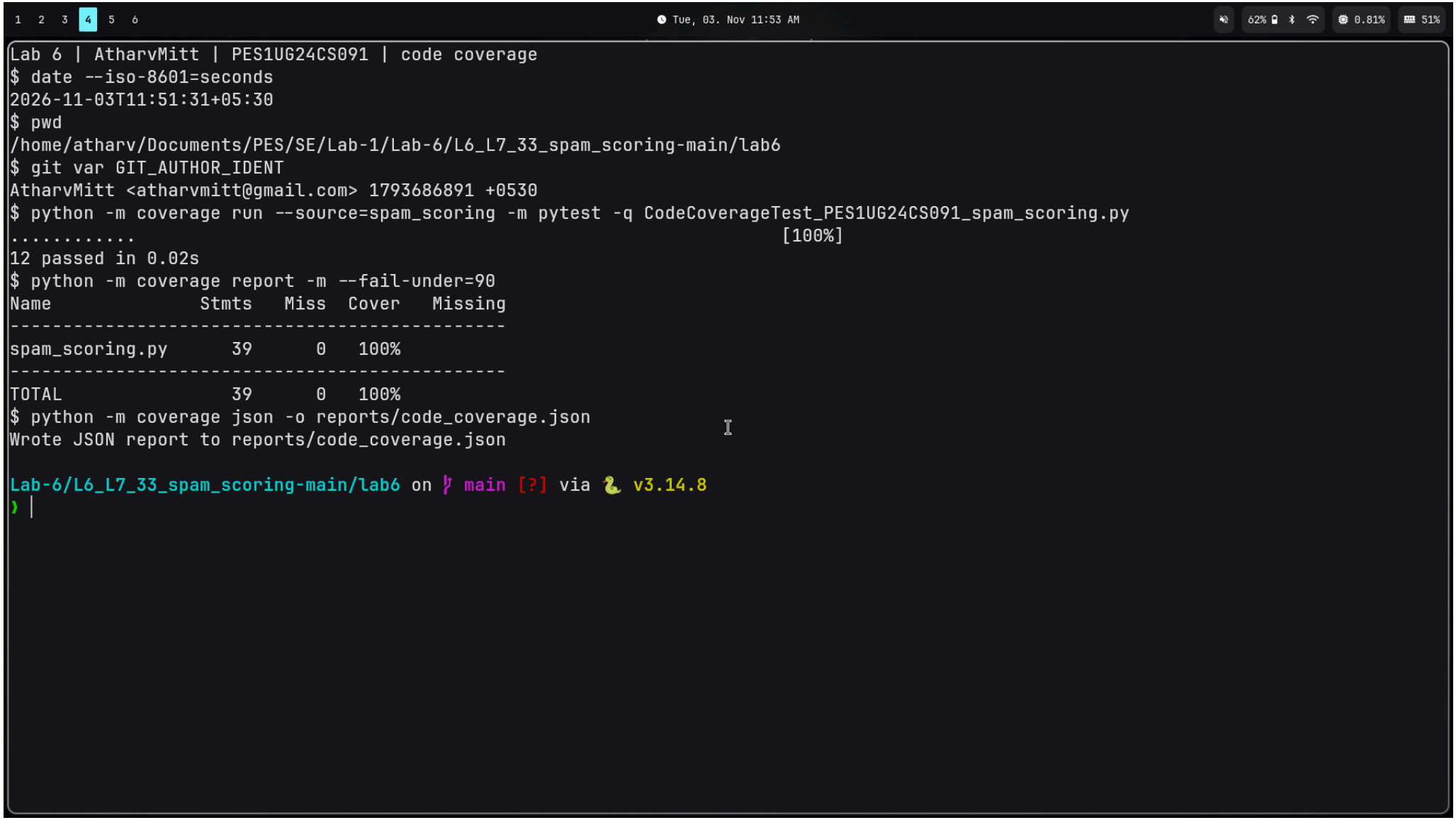
Run from Lab-6/L6_L7_33_spam_scoring-main/lab6. Environment used: Python 3.14.8, coverage 7.17.0 and pytest 9.2.0.

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python -m coverage run --source=spam_scoring -m pytest -q CodeCoverageTest_PES1UG24CS091_spam_scoring.py
python -m coverage report -m
python -m coverage run --branch --source=spam_scoring -m pytest -q BranchCoverageTest_PES1UG24CS091_spam_scoring.py
python -m coverage report -m
python -m pytest -q
```

The default test_spam_scoring.py collects the final branch suite (50 passing cases). Alternatively, bash reports/run_coverage.sh code and bash reports/run_coverage.sh branch reproduce the respective saved reports.

5. Terminal screenshot — code coverage

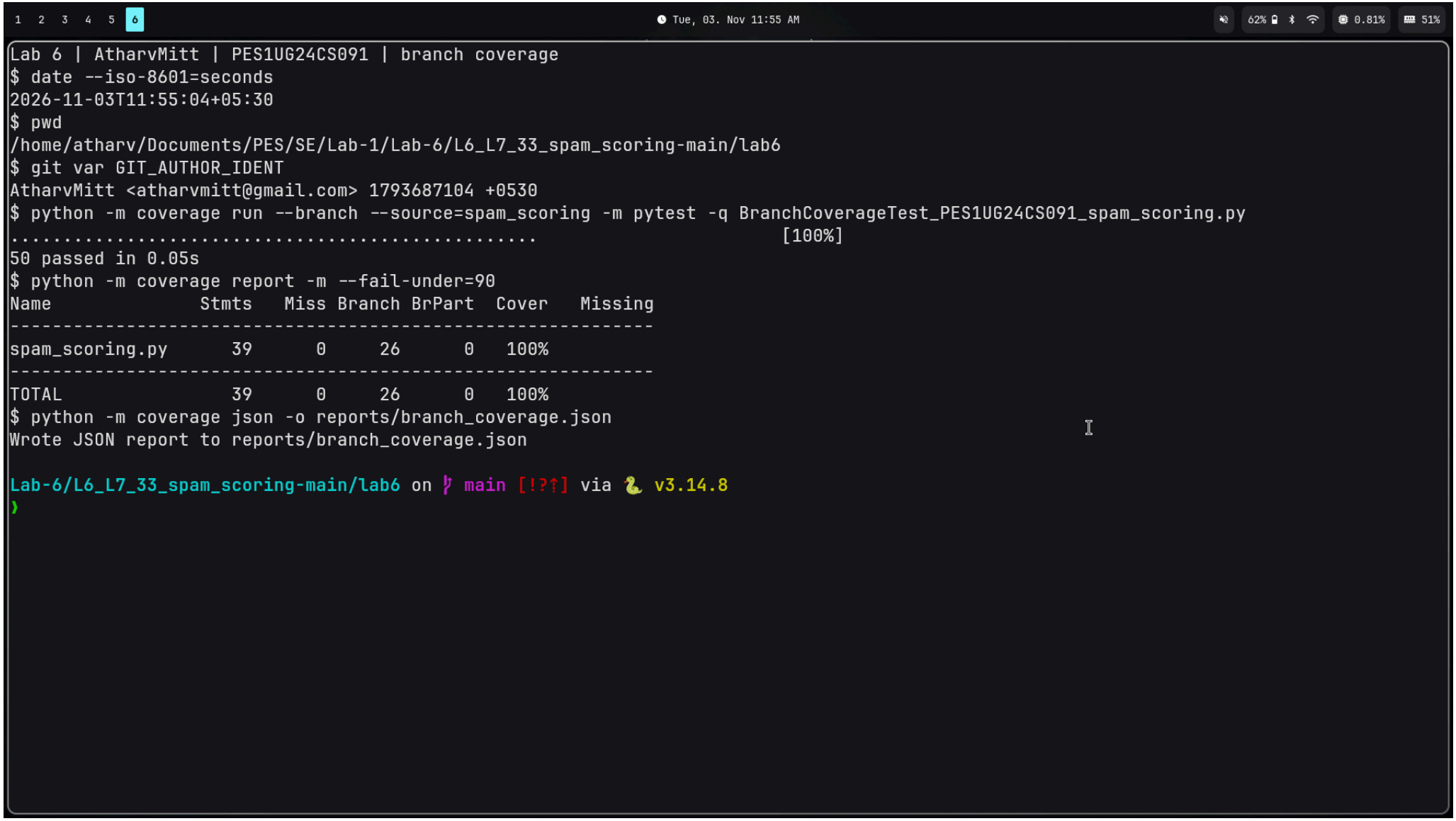
12 tests passed; 39/39 statements; 100% line coverage.



Original file: reports/code_coverage.png | Direct, unedited laptop display capture using grim; actual terminal output.

6. Terminal screenshot — branch coverage

50 tests passed; 39/39 statements and 26/26 branches; 100% coverage.



Original file: reports/branch_coverage.png | Direct, unedited laptop display capture using grim; actual terminal output.